Two-Colour Zig-Zag Numberlink on Rectangles: A Forbidden-Pattern Characterization, a Constructive Polynomial-Time Algorithm, and a Lean Verification

Z. Zzyzek*

Draft, October 2026

Abstract

In Zig-Zag Numberlink, pairs of endpoints in a grid must be joined by vertex-disjoint paths that together visit every cell. The problem is NP-complete when the number of pairs is unbounded (Adcock et al.). For a single pair it is the Hamiltonian path problem in rectangular grid graphs, solved by Itai, Papadimitriou and Szwarcfiter. Two pairs on a rectangle form the paired 2-disjoint path cover problem. We settle the case of two pairs on rectangles with the endpoints anywhere in the grid. If both sides of the rectangle are at least 11, an instance is solvable if and only if no entry of a finite catalogue of forbidden patterns occurs. The catalogue consists of a parity condition, an alternation condition on the border, a handful of local corner and edge patterns, 65 listed corner configurations, and a rule that propagates forced routes out of the corners before testing alternation. If a side is at most 10, a frontier dynamic programming algorithm decides the instance in time linear in the long side. The characterization is constructive. Solutions are built by deleting empty lines, splitting the rectangle by straight cuts, and splicing the pieces back together, in time $O((R+C)\,RC)$. Both directions of the characterization and the correctness of the dynamic programming algorithm are verified in Lean 4. The finite case analyses, which are window certificates for the corner configurations and a check of all irreducible instances with sides 11–22, are carried out by the Lean compiler via `native_decide`. We describe the algorithm, the catalogue, the proof structure, the compromises made in the formalization, and implementations in Python, JavaScript and C that produce identical output.

AI disclosure

Nearly the entirety of the work for this project was done by Claude (Opus 5.5), including writing this paper, writing the reference implementations and writing the Lean 4 proof. I, the listed author, contributed little more than the problem statement and guidance. I have listed myself as an author only to satisfy any requirement that a human author be attached to this work.

1 Introduction

Numberlink is a pencil puzzle. The player is given pairs of numbered cells in a grid and connects each pair by a path along edge-adjacent cells, so that no two paths share a cell. The *zig-zag*

*To the extent possible under law, the author has waived all copyright and related or neighboring rights to this paper and to the accompanying code, proofs and data, under CC0 1.0 Universal (`https://creativecommons.org/publicdomain/zero/1.0/`).

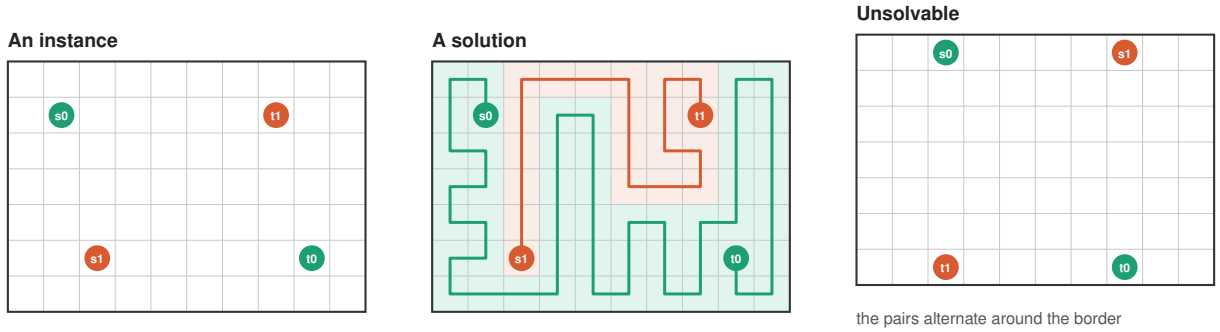


Figure 1: Two-colour Zig-Zag Numberlink on a 7×10 rectangle. Left, an instance with endpoints s_0, t_0 of colour 0 (green) and s_1, t_1 of colour 1 (orange). Middle, a solution. The two vertex-disjoint paths $s_0 \to t_0$ and $s_1 \to t_1$ together visit every cell, and the colour-0 path winds around the colour-1 path. Right, an unsolvable instance. The endpoints alternate in colour around the border, so the two paths would have to cross (catalogue entry T1, Section 5).

variant, popular as the mobile game *Flow Free*, additionally requires every cell of the grid to be used. Adcock et al. [1] showed that Zig-Zag Numberlink is NP-complete when the number of pairs is unbounded.

With a single pair, the problem asks for a Hamiltonian path between two given cells. Shade the cells of the rectangle like a checkerboard. Itai, Papadimitriou and Szwarcfiter (IPS) [4] showed that if the rectangle has an even number of cells, a Hamiltonian path between two cells requires them to have different shades, and if it has an odd number of cells, both must have the majority shade. A pair that meets this condition has a Hamiltonian path unless it belongs to one of three small families of exceptions, which occur only when a side has length 1, 2 or 3.

We study the case where two pairs of endpoints lie on an $R \times C$ rectangle (Figure 1). The instance has endpoints s_0, t_0 of colour 0 and s_1, t_1 of colour 1. A solution is a pair of vertex-disjoint paths $s_0 \to t_0$ and $s_1 \to t_1$ that together cover every cell. In graph theory this is the *paired 2-disjoint path cover* problem. We allow the endpoints anywhere.

With two pairs, one path can block the other. In the right panel of Figure 1, the order of the endpoints around the border rules out a solution on a rectangle of any size. Near a corner, a few endpoints can also force a path through the corner cells, and the forced route can leave the remaining endpoints in alternating order (Section 5). For rectangles with both sides at least 11, we show that four kinds of conditions account for every unsolvable instance. These are the parity of the endpoints, their order around the border, a few patterns of nearby endpoints, and a finite list of configurations near the corners.

Theorem 1 (Main theorem)**.** *Let I be an instance on an $R \times C$ rectangle with $R, C \geq 11$. Then I is solvable if and only if* Passes (I)*, i.e. no entry of the catalogue of Section 5 fires. If $\min(R, C) \leq 10$, solvability is decided by a dynamic programming algorithm in time linear in $\max(R, C)$. In both cases a solution, when one exists, is constructed in polynomial time.*

Since the catalogue is a finite list of patterns, checking it takes time $O(R + C)$ when both sides are at least 11. When a side is at most 10, the dynamic programming algorithm takes time $O(\max(R, C))$, with a constant that depends only on the short side. Together they decide the solvability of any instance in time $O(\max(R, C))$.

Contributions.

- A finite catalogue of forbidden patterns that characterizes solvability for both sides ≥ 11, with the endpoints anywhere in the grid (Section 5).
- A constructive algorithm. Line deletions are undone by a *band extension* lemma that lifts solutions back, straight cuts (*moves*) split the rectangle into pieces whose solutions glue together, and thin rectangles are solved by the plug DP (Sections 6 and A).
- A Lean 4 formalization of both directions and of the thin case (Section 9), with the compromises made explicit.
- Implementations in Python, JavaScript and C, tested against each other, the Lean definitions and exact solvers (Section 8).

2 Definitions

Cells are (r, c) with $0 \leq r < R$ and $0 \leq c < C$. Two cells are adjacent if they differ by one in exactly one coordinate. The instance is *thin* if $\min(R, C) \leq 10$, and the *domain* $\mathcal{D}$ is the set of instances with both sides ≥ 11. Figure 2 illustrates the definitions below.

Lines and gaps. A *line* is a full row or column. It is *free* if it contains no endpoint. A *gap* is a maximal run of consecutive free lines on one axis. An *edge gap* touches the border, and an *internal gap* lies between two lines that contain endpoints (Figure 2a). The reductions of Section 6 delete two lines from a gap.

Shades and signs. A cell is *black* if $r + c$ is even and *white* otherwise. Its sign $\sigma(x)$ is $+1$ if x is black and -1 if white (Figure 2b). We say shade rather than colour, to keep *colour* for the two pairs. Black and white cells alternate along any path, so a path from s to t has sign sum $(\sigma(s) + \sigma(t))/2$. The whole grid has sign sum $RC \bmod 2$. Hence a solution requires $\sum \sigma(\text{endpoint}) = 2 \cdot (RC \bmod 2)$, which is the parity entry P of the catalogue.

One-path pieces. For a rectangle and two cells $s \neq t$, IPS [4] show that a Hamiltonian s–t path exists if and only if two conditions hold. If RC is even, s and t have different shades, and if RC is odd, both are black. In addition, (s, t) is not in one of three explicit families that occur when a side has length 1, 2 or 3. We call such a pair *acceptable* (Figures 2c and 2d). A Hamiltonian path between an acceptable pair can be constructed in linear time.

Cuts and moves. A *cut* is a straight line between two adjacent rows or columns. It splits the rectangle into two *pieces*. Where a solution path crosses the cut, the two cells on either side form a *crossing pair*, and each becomes a *virtual endpoint* of its piece (Figures 2e–2g). The part of the path inside a piece then runs between two endpoints, real or virtual. A piece is a *one-path piece* if it holds the two endpoints of a single path, and a *two-path piece* otherwise. A *move* is a cut together with a choice of crossing pairs. It is *valid* if every piece is OK, meaning that a one-path piece is acceptable and a two-path piece is either thin and solvable or in $\mathcal{D}$ and passing the catalogue. Solutions of the pieces then glue back along the crossing pairs.

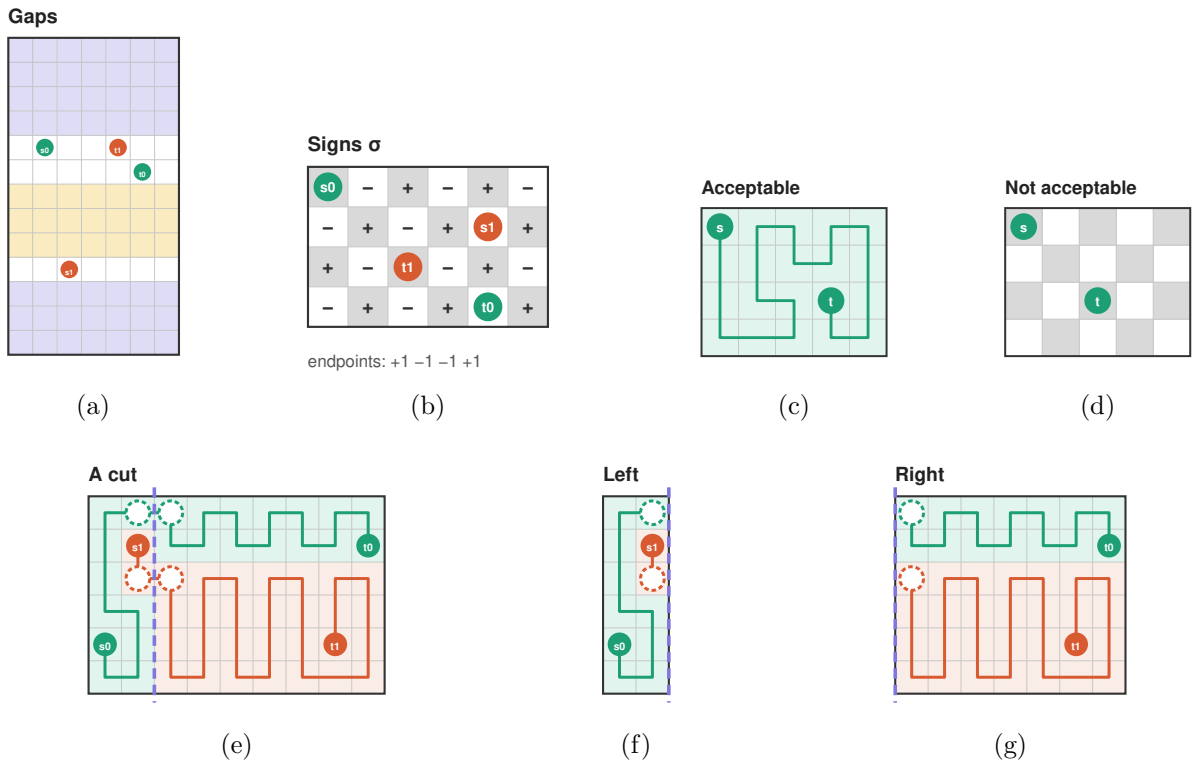


Figure 2: Definitions. (a) A 13×7 instance. Rows 0–3 and 10–12 are free and form edge gaps (purple), and rows 6–8 form an internal gap (yellow). (b) Black cells have sign $\sigma = +1$ and white cells -1. The endpoint signs sum to 0, which is $2 \cdot (RC \bmod 2)$ since $RC = 24$ is even, as parity requires. (c) A one-path piece. The cells s and t have different shades on a 4×5 rectangle, so the pair is acceptable and a Hamiltonian s–t path exists. (d) Both cells are black on a rectangle with an even number of cells, so the pair is not acceptable. (e) A solution and a cut (dashed). Each colour crosses the cut once, and the two crossing pairs are circled. (f), (g) The pieces left and right of the cut, with the cut drawn on the side where they meet. Each crossing cell becomes a virtual endpoint (dashed) of its piece, and the pieces' solutions glue back into (e).

3 Related work

Hamiltonian paths in grid graphs. Itai, Papadimitriou and Szwarcfiter [4] proved the Hamiltonian path problem NP-complete in general grid graphs, which are finite induced subgraphs of the infinite grid and may have holes and irregular boundaries. They also gave the characterization for rectangles that we use for one-path pieces. This paper is about rectangles only. On a rectangle, long runs of free lines can be shortened without changing solvability (Section 6), and the remaining obstructions come from parity, the order of the endpoints along the border, and endpoints near the corners (Section 5). General grid graphs have no such reduction, and for them the problem is NP-complete already for one pair. For one pair on a rectangle the problem is exactly the IPS case.

Numberlink. Adcock et al. [1] proved that Zig-Zag Numberlink is NP-complete. For a single pair the problem is the Hamiltonian path problem, solved on rectangles by IPS. Without the covering requirement, the problem is a vertex-disjoint paths problem. That problem is NP-complete for unbounded k even in planar graphs [7], and polynomial for every fixed k in general graphs by the

Graph Minors theory of Robertson and Seymour [10]. With the covering requirement the paths must also fill the grid, so parity and corner effects become obstructions that do not arise for disjoint paths alone.

Disjoint path covers. In graph theory, Zig-Zag Numberlink is the *paired many-to-many k-disjoint path cover* problem, which asks for k vertex-disjoint paths that join given pairs of vertices and together cover every vertex. It has been studied for many interconnection networks, such as hypercubes, tori, and cylindrical and toroidal grids [9]. Park and Ihm [9] solved the case $k = 2$ on bipartite cylindrical grids, which are rectangles with wraparound along the rows. Above a small size, the only obstructions there are parity and three configurations of the endpoints, and these coincide with our entries T1, T2 and E1. A cylinder has no corners, and the rest of our catalogue concerns the corners of the rectangle. Keshavarz-Kohjerdi [6] stated a characterization for the case where all four endpoints lie on the border of the rectangle.

Frontier dynamic programming algorithms. The dynamic programming algorithm we use for thin rectangles is a transfer-matrix (frontier-based) method over the connectivity states of a boundary. In competitive programming it is known as the *plug DP*. The standard reference is Chen's 2008 paper [2], written in Chinese and summarized in [8]. It introduced the bracket encoding of the frontier that we use, and it handles the endpoints of a path as unpaired ("independent") plugs. In statistical physics the same technique enumerates Hamiltonian circuits, walks and chains on lattices of bounded width [5]. Our DP is this technique with each plug labelled by the colour of its path, and we do not claim novelty for it. Our contribution regarding the DP is its formal verification in Lean (Section 9).

4 Overview of the approach

Thin rectangles, those with $\min(R, C) \leq 10$, are handled exactly by the plug DP (Appendix A). For rectangles with both sides at least 11, every entry of the catalogue other than parity looks only at the endpoints near the four corners, at the order of the endpoints along the border, or at small groups of nearby endpoints. The proof has three parts.

1. *Obstructions* (Theorem B). Each entry of a catalogue of patterns is proved to prevent a solution. Most proofs are short and use parity, the planarity of the two paths, or forced degrees at corners. Some corner configurations are certified by exhaustive local computations.

2. *Reductions* (Theorem A, induction step). If an axis has a long run of free lines, two of them are deleted. The smaller instance passes the catalogue if the larger one does, because each entry sees the same corner windows and the same cyclic order of endpoints along the border. Any solution of the smaller instance extends to the larger one by *band extension*.

3. *Base* (Theorem A, base case). A side of length ≥ 23 always admits a reduction, so the irreducible instances have both sides between 11 and 22. For each passing irreducible instance there is a valid move, a straight cut whose pieces are smaller and OK. This finite statement is checked by computer.

Theorems A and B together give the characterization, and the reductions and moves give the construction.

5 The catalogue of forbidden features

The catalogue is closed under the eight symmetries of the rectangle, under swapping the colours, and under swapping s with t within a colour. We list entries up to these symmetries. Corner entries are evaluated in the eight *corner frames* (four corners, two orientations), with frame coordinates (i, j) counted from the corner. Below, γ is either colour and $\bar{\gamma} = 1 - \gamma$ is the other one. Entry names are set in sans-serif (R, C) to distinguish them from the side lengths R and C.

Entry	Fires when
P	$\sum \sigma(\text{endpoint}) \neq 2 \cdot (RC \bmod 2)$ (the parity of two covering paths)
T1	all endpoints are on the border and their colours alternate $0, 1, 0, 1$ around it
T2	the endpoints fill a 2×2 square with each colour on a diagonal
L1	an endpoint's in-grid neighbours are all endpoints of the other colour
L6	two corners are closed off, one by each colour, each being an empty corner cell whose two neighbours are endpoints of one colour
L2–L5	small corner patterns, for example L2 with $(0, 1)$ of colour γ, $(1, 0)$ of colour $\bar{\gamma}$, and $(0, 0)$ empty
E1	along an edge, $(0, \ell)$ and $(1, \ell{+}1)$ have colour γ and $(1, \ell{+}2)$ and $(0, \ell{+}3)$ have colour $\bar{\gamma}$
B	a listed corner triple occurs (21 triples, or 5 when both sides are odd), with the fourth endpoint on the border outside the 6×6 corner window
C, D	the four endpoints form one of 36 listed corner configurations (RC even) or 3 (RC odd) within an 8×8 corner window
R	after forced corner routes (rules R1–R3), the effective endpoints alternate along the border of the remaining region (Figure 4)

Table 1: The catalogue. The lists for B, C and D are in Appendix C.

The R entry. Some corner configurations force a path through the corner cells (Figure 4). The rules below are stated in a corner frame.

R1 An endpoint at $(0, 1)$, with $(0, 0)$ and $(1, 0)$ not endpoints. The corner cell $(0, 0)$ has only two neighbours, so the path runs $(0, 1) \to (0, 0) \to (1, 0)$. The endpoint effectively moves to $(1, 0)$, and $(0, 0)$ and $(0, 1)$ are settled.

R2 Endpoints of one colour at $(0, 0)$ and $(1, 1)$, with $(0, 1)$, $(1, 0)$, $(0, 2)$ and $(2, 0)$ not endpoints. The paths are forced around the 2×2 corner square, and the endpoints effectively move to $(0, 2)$ and $(2, 0)$.

R3 Endpoints of different colours at $(1, 2)$ and $(2, 1)$, with $(0, 0)$–$(0, 3)$, $(1, 0)$, $(1, 1)$, $(2, 0)$ and $(3, 0)$ not endpoints. The two paths fill the corner, the endpoints effectively move to $(0, 3)$ and $(3, 0)$, and eight cells are settled.

The rules are applied once per corner frame in a fixed order (at each frame R3, then R2, then R1), skipping a rewrite that would overlap cells already settled. R fires if at least one rewrite applied and the four effective endpoints are distinct cells on the boundary of the remaining region whose colours alternate along it. This is T1 for the remaining region.

Where the entries come from. P and T1 are the classical obstructions. Each path alternates black and white cells, which gives parity, and two disjoint paths between alternating border points

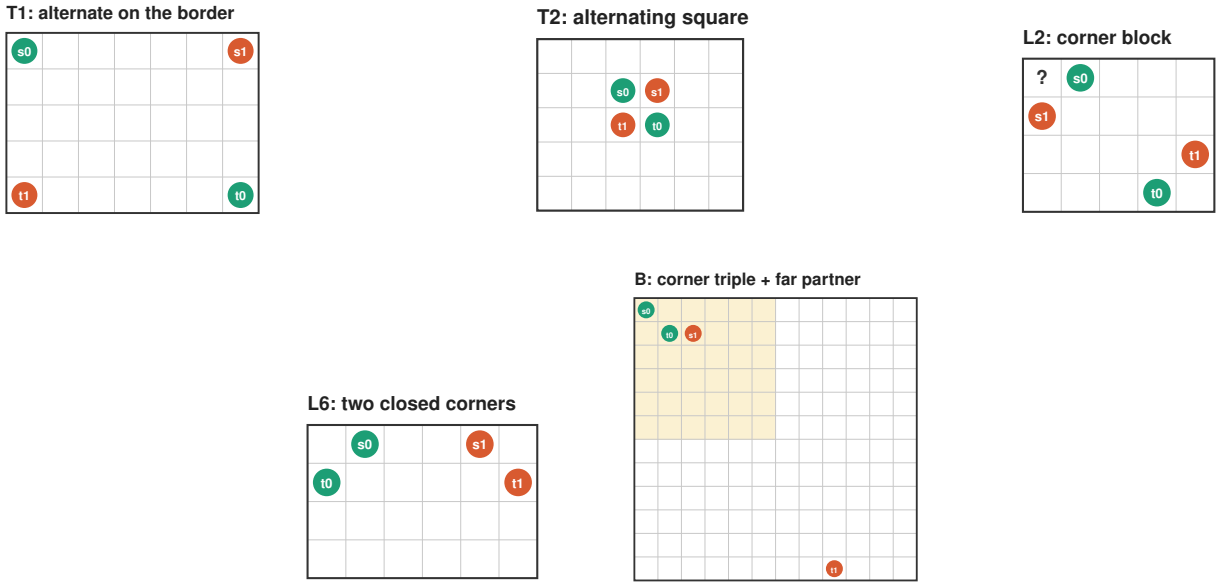


Figure 3: Entries T1, T2, L2, L6 and B. In T2 the endpoints fill a 2×2 square with each colour on a diagonal, so the two paths would have to cross inside the square. In L2 the corner cell (marked ?) can only be reached from two endpoints of different colours. In B the 6×6 corner window is shaded.

would cross, which gives planarity. T2, L1–L6 and E1 express forced degrees near endpoints and corners. B, C and D are corner configurations that are unsolvable for reasons that are local but not short. Each was found by exhaustive search on corner windows, and each is certified by a window computation (Section 7).

6 The algorithm

6.1 Reductions and band extension

On an axis of length $n \geq 13$, the following reductions delete two free lines d and $d+1$.

(S) A *strip* applies to an edge gap of at least 4 lines and deletes the outermost two.

(K) A *compression* applies to an internal gap of at least 10 lines and deletes two from its middle.

(I) An *interior compression* applies to an internal gap of at least 3 lines containing d and $d+1$ with $8 \leq d \leq n-10$.

Lemma 2 (Band extension)**.** *If I' is solvable and row r of I' is free, then inserting two empty rows directly below r gives a solvable instance I. A solution of I is obtained from one of I' in time $O(RC)$.*

Construction. Stretch every path edge crossing below row r through the two new rows. The new rows split into blocks between consecutive crossings, and each crossing absorbs the block on one of its sides by a U-shaped detour. The remaining block is absorbed by a *square flip*, in which a path edge (r, a)–$(r, a{+}1)$ inside it is replaced by the block's boundary cycle. Such an edge exists by counting. The m crossing cells of row r supply at most m horizontal edges, while each of the

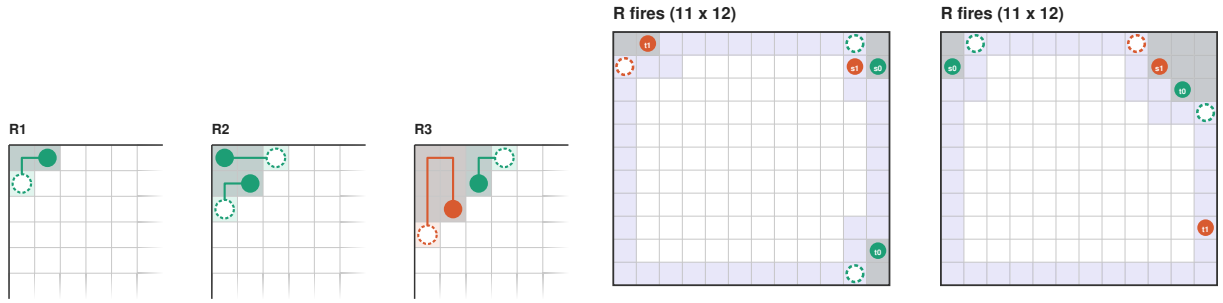


Figure 4: The R entry. (a)–(c) Rules R1–R3 at the top-left corner of the grid, which continues to the right and below. Green is a colour γ and orange the other colour $\bar{\gamma}$. The forced path pieces are drawn, settled cells are grey, and dashed circles are the effective endpoints. (d), (e) Two 11×12 instances on which R fires and no other entry does. Neither has a solution. After the forced routes, the four effective endpoints lie on the border of the remaining region (shaded purple) and alternate in colour along it. This is the T1 condition for that region, so the two paths would have to cross. In (d) rule R1 applies at three corners, and in (e) rule R3 settles eight cells at the top-right corner and R1 settles two at the top-left.

$m + 1$ runs of non-crossing cells needs one, because its cells have degree two and no vertical edge downward. □

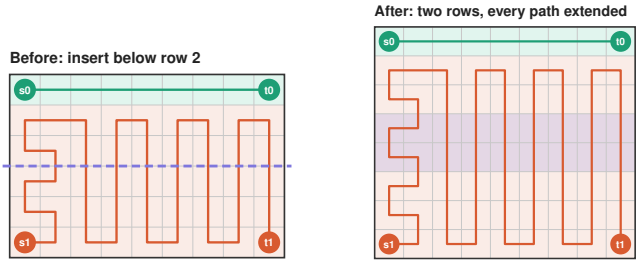


Figure 5: Band extension. Two rows are inserted below row 2.

6.2 Moves

A move cuts the rectangle between rows $p-1$ and p, up to symmetry. The five types differ in how the endpoints fall (Figure 6). In a *strip* all endpoints are on one side, and the other side is empty, has even area and at least two lines. Its block is spliced into a path running along the cut. In a *1/3* move one endpoint is alone, and its piece is a one-path piece to its crossing. A *2/2 cross* has one endpoint of each colour on each side and two crossings. In a *2/2 same* move each side holds one whole pair, giving two one-path pieces. An *excursion* also has one whole pair on each side, and one colour crosses the cut twice. The pieces' solutions glue along the crossing edges.

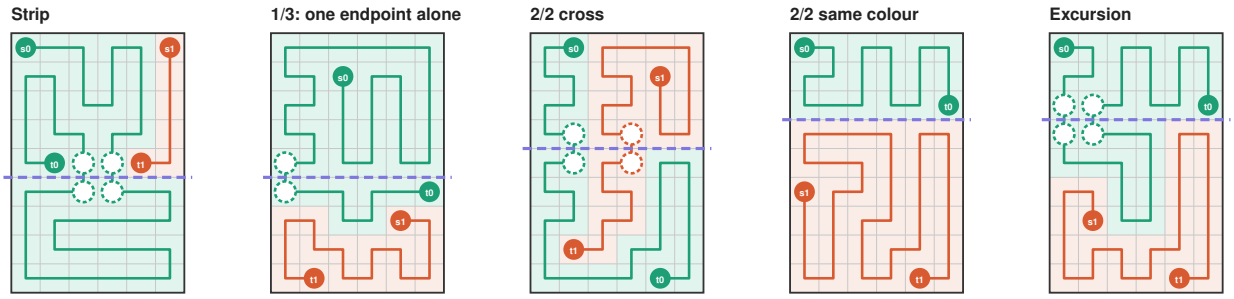


Figure 6: The five move types. Each cuts the rectangle along the dashed line, and the dashed circles mark crossing pairs, whose cells become virtual endpoints of the pieces. In a *strip* all four endpoints lie on one side, and the other side is an empty block of even area, spliced into a path that runs along the cut. In a *1/3* move one endpoint is alone on its side. Its path crosses the cut once, so that side is a one-path piece from the endpoint to its crossing cell, and the other side is a two-path piece. In a *2/2 cross* each side holds one endpoint of each colour. Each path crosses once, and both sides are two-path pieces. In a *2/2 same* move each side holds both endpoints of one colour. No path crosses, and both sides are one-path pieces. In an *excursion* each side holds one whole pair as in 2/2 same, but one path (here colour 0) leaves its side and returns, crossing twice.

Algorithm 1 Decision

Input: instance $I = (R, C, s_0, t_0, s_1, t_1)$ with four distinct endpoints
Output: whether I is solvable

1: **if** $\min(R, C) \leq 10$ **then return** PlugDPFeasible(I) ▷ Appendix A
2: **else return not** Fires(I) ▷ Section 5

6.3 Pseudocode

Algorithm 1 decides an instance, Algorithm 2 constructs a solution, and Algorithm 3 is the band extension used to undo a reduction. The catalogue test Fires is written out in Appendix B (Algorithm 5). Each entry's test is a constant-size pattern check (Table 1 and Appendix C), except for T1, E1 and R, which scan the border. The test takes time $O(R + C)$.

6.4 Complexity

Each reduction removes two lines, so at most $(R + C)/2$ reductions occur. Each costs a catalogue check ($O(R + C)$) and a band extension ($O(RC)$). Moves occur only on rectangles with both sides at most 22, in constant time. The total is $O((R + C)\, RC)$ for $R, C \geq 11$, plus the plug DP, which is linear in the long side for a fixed short side.

6.5 A worked example

Consider the 16×13 instance of Figure 7. Rows 0–3 are free, so the strip reduction deletes rows 0 and 1. The 14×13 result has no reduction, and strip moves cut off a 2×13 block below and a 12×2 block on the right. The 12×11 core is split by a horizontal 2/2-cross move into two thin 6×11 pieces, which the plug DP solves. The blocks are spliced back, and the two deleted rows are reinserted by band extension.

Algorithm 2 Construction

function SOLVE(I)
  if $\min(R, C) \le 10$ **then return** PLUGDPSOLVE(I)
  if FIRES(I) **then return** NONE
  if $(axis, d) \leftarrow$ FINDREDUCTION(I) succeeds **then**
    $I' \leftarrow I$ with lines $d, d+1$ on $axis$ deleted
    $S' \leftarrow$ SOLVE(I')
    return BANDEXTEND(S', $axis$, d) ▷ Lemma 2
  if a cut separates the two pairs and both one-path pieces are acceptable **then**
    return the two Hamiltonian paths (IPS construction)
  $M \leftarrow$ FINDMOVE(I) ▷ exists by Theorem A, and here $R, C \le 22$
  solve each piece of M (by IPS, the plug DP, or SOLVE)
  return the pieces' solutions glued along M's crossings

function FINDREDUCTION(I)
  for $axis \in \{\text{rows}, \text{columns}\}$ with length $n \ge 13$ **do**
    if lines 0..3 are free **then return** $(axis, 0)$
    if lines $n{-}4..n{-}1$ are free **then return** $(axis, n-2)$
    if lines $\ell..\ell{+}9$ are free, with endpoints on both sides **then return** $(axis, \ell+4)$
    if $8 \le d \le n{-}10$, lines $d, d{+}1$ and $d{-}1$ or $d{+}2$ free, endpoints on both sides **then**
      return $(axis, d)$
  return NONE

function FINDMOVE(I)
  for $m = 0, 1, \ldots, 10$ **do** ▷ width of the widest thin piece, 0 for none
    for each move type, each of the 8 orientations, each endpoint relabelling **do**
      for each cut position p and crossing choice whose widest thin piece has width m **do**
        if every piece is OK (narrower piece checked first) **then return** the move

7 Proof outline

Theorem 3 (Theorem B, soundness of the catalogue)**.** *If an entry fires on an instance in $\mathcal{D}$, the instance is unsolvable.*

P, T1, T2, L1–L6, E1 and R have direct proofs, using parity, the fact that two disjoint paths between alternating border points of a disc must cross, and forced degrees. B, C and D are certified by *window certificates*. A dynamic programming algorithm over the corner window enumerates all ways a solution can look inside it, and an outside test shows that none of them can be completed outside the window, for any rectangle in $\mathcal{D}$. The outside test uses parity, a non-crossing perfect matching of the boundary crossings, and the structure of the paths.

Theorem 4 (Theorem A, completeness of the catalogue)**.** *Every instance in $\mathcal{D}$ that passes the catalogue is solvable.*

The proof is by induction on the area. If a reduction applies, the smaller instance passes because each entry's view is unchanged, so it is solvable by induction, and band extension lifts its solution.

Algorithm 3 Band extension (Lemma 2), rows

Input: a solution P of I' and a free row r
Output: a solution of I' with two empty rows $n_1 = r{+}1$, $n_2 = r{+}2$ inserted

shift every cell below row r down by two
$X \leftarrow$ columns c where P uses the edge (r, c)–$(r{+}3, c)$ ▷ crossings
for $c \in X$ **do** replace the edge by $(r, c), (n_1, c), (n_2, c), (r{+}3, c)$
let $B_0, \ldots, B_{|X|}$ be the column blocks of rows n_1, n_2 between crossings
$j \leftarrow$ an index with $B_j = \emptyset$, or else one whose block contains a path edge (r, a)–$(r, a{+}1)$
for the i-th crossing c **do** absorb B_{i-1} if $i \le j$, else B_i, by a U-detour replacing (n_1, c)–(n_2, c)
if $B_j \neq \emptyset$ **then** replace (r, a)–$(r, a{+}1)$ by the boundary cycle of B_j opened at (n_1, a)–$(n_1, a{+}1)$

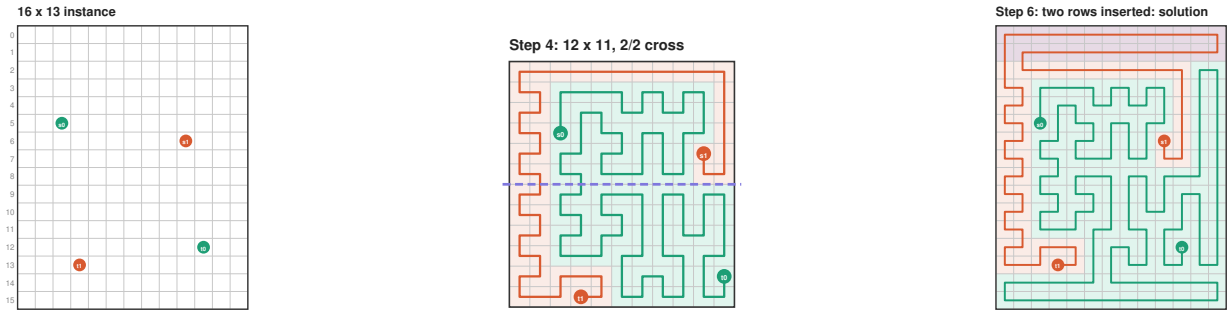


Figure 7: A 16×13 instance, the 12×11 core with its 2/2-cross move, and the solution.

Otherwise both sides are at most 22, since a side ≥ 23 always has a reduction. For these finitely many instances a computer check shows that a valid move exists. Its thin pieces are solvable, its other two-path pieces are smaller passing instances in $\mathcal{D}$ and solvable by induction, and the pieces glue.

8 Implementations

We provide libraries in Python (the reference), JavaScript and C, each with a command-line program, in the repository `https://github.com/zzyzek/R2K2ZZN`. All three implement Algorithms 1–3 and the plug DP with the same iteration orders, and produce byte-identical output, including the paths. They were tested as follows.

- The catalogue of each implementation agrees with the Lean definition on 60,000 random instances with sides 2–24 and endpoints clustered near corners.
- The three implementations agree on 450 random instances with sides 3–45, and every reported solution is verified.
- The C implementation solves 20,000 random instances with sides 11–60 in about 8 seconds, with DECIDE and SOLVE agreeing on every instance.
- On 300 random instances with sides 11–12, DECIDE agrees with an independent exact solver.

Each library also offers an optimized solver for thin instances. It cuts long thin rectangles into pieces at guessed splits and falls back to the plug DP when a guess fails. It is exact, and on random 10-wide instances it is typically three orders of magnitude faster.

9 Verification in Lean

The development is about 27,000 lines of Lean 4 [3] with Mathlib [11]. The trusted statement is a short file defining instances, paths and solutions. The main results are the following.

- `passes_iff_solvable` states that for $R, C \geq 11$ and a well-formed instance, $\textsc{Passes}(I) \leftrightarrow \text{Solvable}(I)$.
- `thinBF_iff` states that the proof-oriented plug DP accepts if and only if the instance is solvable, for every rectangle. It uses only the standard axioms.
- `solvableB_iff` states that a decision procedure for every rectangle is correct.

Here Passes is a computable Boolean function. The theorem is about it as defined, so no informal transcription of the catalogue needs to be trusted. The IPS characterization is formalized as well and used for one-path pieces.

Compromises.

1. *Computation by `native_decide`.* The window certificates (10 theorems) and the base case (144 theorems, one per box of sides 11–22) are evaluated by the Lean compiler rather than the kernel. This adds 154 axioms that trust the compiler and the generated native code. Kernel evaluation is not practical. On a 6×6 window the kernel exceeded 16 GB after 20 minutes on a computation the compiled code does in 3 seconds, and an 8×8 window reaches 2.3×10^5 states.
2. *Symmetry handling.* The base case checks one representative per orbit of the 64-element symmetry group (geometric symmetries, colour swap, reversing either path). Instead of proving that the catalogue is invariant under reflections, the computation checks that the representative or one of its reflections passes. Invariance under the other symmetries is proved. The search region is about 2.8×10^{10} instances, and the check takes about 16 CPU-hours (4 hours on 4 threads).
3. *Checked search instead of verified search.* The move search is a Boolean function proved sound, so a move it finds is valid. Thin pieces are solved by an unverified solver, and the solutions are checked by a verified checker.
4. *Precompiled modules.* Without `precompileModules`, `native_decide` interprets the project's functions and is about 400× slower.
5. *Running time.* The polynomial bound is not formalized, since Lean has no standard cost model short of Turing machines. The theorems certify what is computed, and the running-time analysis is on paper.
6. *Thin case.* The verified plug DP uses a proof-friendly state encoding, with sorted lists of pieces coded by their ends, rather than the bracket encoding of the implementations.

10 Discussion

The characterization gives a linear-time decision procedure for two pairs on rectangles with both sides at least 11, while the problem is NP-complete for an unbounded number of pairs. Open

questions include three or more pairs on rectangles, where corner interactions multiply, and two pairs on other regions such as solid grid graphs. The corner configurations of the catalogue (B, C, D) were found by computer. A conceptual description of them would shorten the proof and might allow kernel-checked certificates.

Code and proofs. The Lean development, the libraries in Python, JavaScript and C, a prose account of the proof, and the source of this paper are available at `https://github.com/zzyzek/R2K2ZZN`. They are dedicated to the public domain under CC0 1.0 Universal.

References

[1] Aaron B. Adcock, Erik D. Demaine, Martin L. Demaine, Michael P. O'Brien, Felix Reidl, Fernando Sánchez Villaamil, and Blair D. Sullivan. Zig-zag Numberlink is NP-complete. *Journal of Information Processing*, 23(3):239–245, 2015. arXiv:1410.5845.

[2] Danqi Chen. Dynamic programming problems based on connectivity state compression. Paper of the Chinese National Olympiad in Informatics (NOI) national training team, 2008. In Chinese.

[3] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635, 2021.

[4] Alon Itai, Christos H. Papadimitriou, and Jayme Luiz Szwarcfiter. Hamilton paths in grid graphs. *SIAM Journal on Computing*, 11(4):676–686, 1982.

[5] Jesper Lykke Jacobsen. Exact enumeration of Hamiltonian circuits, walks and chains in two and three dimensions. *Journal of Physics A: Mathematical and Theoretical*, 40(49):14667–14678, 2007.

[6] Fatemeh Keshavarz-Kohjerdi. Paired many-to-many 2-disjoint path covers in meshes. *IEEE Transactions on Parallel and Distributed Systems*, 35(10):1854–1866, 2024.

[7] James F. Lynch. The equivalence of theorem proving and the interconnection problem. *ACM SIGDA Newsletter*, 5(3):31–36, 1975.

[8] OI Wiki contributors. Plug DP. `https://oi-wiki.org/dp/plug/`, 2026. In Chinese; accessed October 2026.

[9] Jung-Heum Park and Insung Ihm. Many-to-many two-disjoint path covers in cylindrical and toroidal grids. *Discrete Applied Mathematics*, 185:168–191, 2015.

[10] Neil Robertson and Paul D. Seymour. Graph minors. XIII. the disjoint paths problem. *Journal of Combinatorial Theory, Series B*, 63(1):65–110, 1995.

[11] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*, pages 367–381, 2020.

A The plug DP

Transpose so that the grid has $W \le L$ rows. Cells are processed column by column, top to bottom. The *frontier* after some cells consists of $W + 1$ edges between processed and unprocessed cells, one per row plus the edge below the current cell (Figure 8). A state assigns each frontier edge a 3-bit code. The edge is unused, or it is an *anchor* of colour 0 or 1, which is a piece ending at an endpoint, or it is an *open* or *close* bracket of colour 0 or 1. A bracket pair is a piece with both ends on the frontier, and the pairs match like parentheses because pieces cannot cross. This is the bracket encoding of Chen [2], with each plug labelled by its colour.

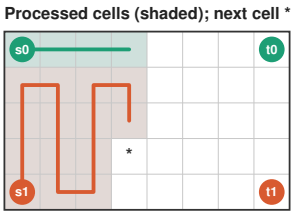


Figure 8: Processed cells (shaded) and the next cell ($*$).

Algorithm 4 Plug DP (decision)

function PLUGDPFEASIBLE(I) ▷ $W \le L$ after transposition
  $S \leftarrow$ {all-empty}
  for column $c = 0, \dots, L-1$ **do**
    if $c > 0$ **then** shift every state by one slot
    for row $r = 0, \dots, W-1$ **do**
      $S \leftarrow \bigcup_{s \in S} \text{STEP}(s, r, c)$
  return all-empty $\in S$

function STEP(s, r, c) ▷ slot r is the plug from above, $r{+}1$ the plug from the left
  $b \leftarrow 1$ if (r, c) is an endpoint of colour γ, else 2
  $h \leftarrow$ the number of incoming plugs
  if $h > b$ or $b - h$ exceeds the free directions (right, down) **then return** $\emptyset$
  if $h = 0$ **then**
    at an endpoint, start a new anchor of colour γ, going right or down
    otherwise start a new bracket pair of either colour, going right and down
  else if $h = 1$ **then**
    continue the plug right or down, or end it if (r, c) is an endpoint of its colour
    (ending an anchor completes its path, and ending a bracket makes its partner an anchor)
  else ▷ join two plugs of the same colour
    two anchors complete the path, and an anchor and a bracket make the partner an anchor
    a close then an open merge the pieces, and an open then a close form a cycle (rejected)
    two opens or two closes change the bracket type of the outer partner
  return the resulting states, with outgoing plugs in slot r (right) and $r{+}1$ (down)

Construction. The forward pass stores one checkpoint, the state set, per column. A backward pass re-runs each column from its checkpoint, recording predecessors and keeping only states whose processed slots agree with the known end-of-column state. It then traces back the chosen edges. The number of states is bounded by the number of valid frontiers (about 10^5 for $W = 10$), so the running time is $O(L \cdot W \cdot \#\text{states})$.

Correctness. By induction over the cells, the reachable states are exactly the signatures of partial solutions on the processed cells. A signature records the used frontier edges, their pairing, and the colours of pieces ending at endpoints. Hence the empty final state is reachable if and only if a solution exists. Lean verifies this for an equivalent DP with a different encoding.

B The catalogue test

Algorithm 5 tests the entries of Table 1 in the order used by the implementations and the Lean definition, and returns the first one that fires.

Algorithm 5 The catalogue test

function FIRES(I) ▷ the first entry that fires, or NONE
  if $\sum \sigma(\text{endpoint}) \neq 2 \cdot (RC \bmod 2)$ **then return** P
  if all endpoints are on the border and their colours alternate around it **then return** T1
  if the endpoints fill a 2×2 square, each colour on a diagonal **then return** T2
  if an endpoint's neighbours are all endpoints of the other colour **then return** L1
  if two corners are closed off by endpoints, one corner by each colour **then return** L6
  for each of the 8 corner frames f **do**
    if a pattern L2–L5 occurs at the corner, or E1 along the frame's first edge **then**
      return that entry
    if a listed B triple occurs in f and the fourth endpoint is on the border outside the 6×6 window **then return** B
    if $R, C \geq 10$ and the endpoints form a listed C (RC even) or D (RC odd) configuration in f **then return** it
  apply rules R1–R3 at each corner frame (R3, R2, R1, without overlaps), giving effective endpoints E' and settled cells S
  if a rule applied and E' are four distinct cells on the boundary of the grid minus S, with colours alternating along it **then return** R
  return NONE

C The listed corner configurations

Cells are written ij for frame coordinates (i, j).
B, for RC even. Each triple $(p\,q\,r)$ has colour γ at p and q and colour $\bar{\gamma}$ at r.
(00 11 12), (01 02 11), (01 02 12), (01 03 11), (01 04 11), (01 04 12), (01 05 11), (01 12 11), (01 12 13), (01 12 22), (01 12 31), (01 13 03), (01 21 13), (01 21 22), (01 21 31), (02 21 10), (03 11 10), (04 11 10), (04 21 10), (04 21 12), (05 11 10).
B, for R and C odd.
(00 11 12), (00 11 22), (01 02 11), (01 04 11), (04 11 10).

C, for RC even. Each entry is {one pair | the other pair}.
{01 03 | 11 13}, {01 06 | 07 11}, {01 06 | 11 70}, {01 06 | 12 60}, {01 07 | 11 60}, {01 13 | 11 12}, {01 13 | 12 22}, {01 13 | 12 31}, {01 22 | 11 12}, {01 22 | 12 31}, {01 22 | 13 30}, {01 70 | 11 60}, {02 11 | 21 30}, {02 20 | 12 21}, {03 21 | 31 40}, {06 21 | 12 60}, {12 22 | 13 21}, {12 31 | 13 21}, {01 12 | 04 22}, {01 12 | 04 31}, {01 12 | 13 20}, {01 12 | 13 40}, {01 12 | 20 22}, {01 12 | 22 40}, {01 12 | 31 40}, {01 13 | 03 04}, {01 13 | 03 20}, {01 13 | 03 40}, {01 21 | 02 22}, {01 21 | 02 31}, {01 21 | 04 13}, {01 21 | 04 22}, {01 21 | 04 31}, {01 21 | 13 40}, {01 21 | 22 40}, {01 21 | 31 40}.
D, for RC odd.
{00 01 | 11 20}, {00 14 | 02 13}, {00 23 | 02 13}.